

SMART CONTRACT AUDIT REPORT

for

2Factor Finance

Prepared By: Xiaomi Huang

PeckShield

July 16, 2026

Document Properties

Client	Fragments, Inc.
Title	Smart Contract Audit Report
Target	2Factor
Version	1.0
Author	Xuxian Jiang
Auditors	Matthew Jiang, Xuxian Jiang
Reviewed by	Xiaomi Huang
Approved by	Xuxian Jiang
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	July 16, 2026	Xuxian Jiang	Final Release
1.0-rc1	July 15, 2026	Xuxian Jiang	Release Candidate #1

Contact

For more information about this document and its contents, please contact PeckShield Inc.

Name	Xiaomi Huang
Phone	+86 183 5897 7782
Email	contact@peckshield.com

Contents

1 Introduction **4**
- 1.1 About 2Factor Finance . . . 4
- 1.2 About PeckShield . . . 5
- 1.3 Methodology . . . 5
- 1.4 Disclaimer . . . 7

2 Findings **9**
- 2.1 Summary . . . 9
- 2.2 Key Findings . . . 10

3 Detailed Results **11**
- 3.1 Possible Stale Perp State Use in PerpVaultHelpers::flashMintPerpByValue() . . . 11
- 3.2 Missing Swap Leftover Refund in UniswapV4CbBtcUsdcStrategy . . . 12
- 3.3 Improved Validation of Function Arguments . . . 14
- 3.4 Improved Precision By Multiplication And Division Reordering . . . 15
- 3.5 Revisited Tranche Set Management Logic in PerpTranche . . . 16
- 3.6 Trust Issue of Admin Keys . . . 17

4 Conclusion **19**

References **20**

1 Introduction

Given the opportunity to review the design document and related smart contract source code of the 2Factor Finance protocol, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of smart contracts can be further improved due to the presence of several issues related to either security or performance. This document outlines our audit results.

1.1 About 2Factor Finance

2Factor Finance is an innovative protocol for perpetual tranches (perp-tranches). The system creates senior (Sr) and junior (Jr) tranches from underlying assets, wrapped as perpetual ERC20 notes with automatic rollover and rebalancing mechanics. It enables users to gain leveraged or de-leveraged exposure to underlying assets through a two-tranche bond system. Users can mint perpetual senior notes (PerpSr) for lower-volatility exposure or perpetual junior notes (PerpJr) for leveraged exposure, backed by dynamically managed reserves of tranched bonds. The basic information of the audited protocol is as follows:

Table 1.1: Basic Information of The 2Factor Finance Protocol

Item	Description
Client	Fragments, Inc.
Website	https://2factor.finance
Type	EVM Smart Contract
Platform	Solidity
Audit Method	Whitebox
Latest Audit Report	July 16, 2026

In the following, we show the Git repository of reviewed files and the commit hash value used in

this audit.

- https://github.com/fragmentsorg/2factor.git (303a427)

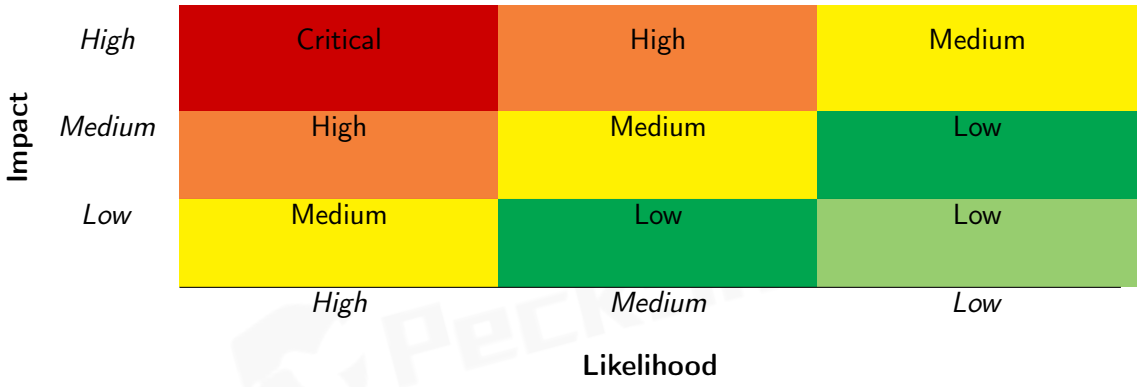
And here is the commit ID after all fixes for the issues found in the audit have been checked in:

- https://github.com/fragmentsorg/2factor.git (a22ab82)

1.2 About PeckShield

PeckShield Inc. [11] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (https://t.me/peckshield), Twitter (http://twitter.com/peckshield), or Email (contact@peckshield.com).

Table 1.2: Vulnerability Severity Classification



1.3 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Table 1.3: The Full List of Check Items

Category	Check Item
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact and can be classified into four categories accordingly, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

To evaluate the risk, we go through a list of check items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the contract is considered safe regarding the check item. For any discovered issue, we might further deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [9], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings.

1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

Category	Summary
Configuration	Weaknesses in this category are typically introduced during the configuration of the software.
Data Processing Issues	Weaknesses in this category are typically found in functionality that processes data.
Numeric Errors	Weaknesses in this category are related to improper calculation or conversion of numbers.
Security Features	Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)
Time and State	Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.
Error Conditions, Return Values, Status Codes	Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.
Resource Management	Weaknesses in this category are related to improper management of system resources.
Behavioral Issues	Weaknesses in this category are related to unexpected behaviors from code that an application uses.
Business Logics	Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.
Initialization and Cleanup	Weaknesses in this category occur in behaviors that are used for initialization and breakdown.
Arguments and Parameters	Weaknesses in this category are related to improper use of arguments or parameters within function calls.
Expression Issues	Weaknesses in this category are related to incorrectly written expressions within code.
Coding Practices	Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained.

2 Findings

2.1 Summary

Here is a summary of our findings after analyzing the design and implementation of the 2Factor Finance protocol. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

Severity	# of Findings	
Critical	0	
High	0	
Medium	1	■
Low	4	■■■■
Informational	1	■
Total	6	

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of, while others refer to unusual interactions among multiple contracts. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities that need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in Section 3.

2.2 Key Findings

Overall, these smart contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 1 medium-severity vulnerability, 4 low-severity vulnerabilities, and 1 informational recommendation.

Table 2.1: Key 2Factor Audit Findings

ID	Severity	Title	Category	Status
PVE-001	Low	Possible Stale Perp State Use in PerpVaultHelpers::flashMintPerpByValue()	Business Logic	Resolved
PVE-002	Low	Missing Swap Leftover Refund in UniswapV4CbBtcUsdcStrategy	Business Logic	Resolved
PVE-003	Low	Improved Validation of Function Arguments	Coding Practices	Resolved
PVE-004	Low	Improved Precision By Multiplication/Division Reordering	Numeric Errors	Resolved
PVE-005	Informational	Revisited Tranche Set Management Logic in PerpTranche	Business Logic	Resolved
PVE-006	Medium	Trust on Admin Keys	Security Features	Resolved

Besides recommending specific countermeasures to mitigate these issues, we also emphasize that it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms need to kick in at the very moment when the contracts are being deployed in mainnet. Please refer to Section 3 for details.

3 Detailed Results

3.1 Possible Stale Perp State Use in PerpVaultHelpers::flashMintPerpByValue()

- ID: PVE-001
- Severity: Low
- Likelihood: Medium
- Impact: Low
- Target: `PerpVaultHelpers`
- Category: Business Logic [7]
- CWE subcategory: CWE-770 [4]

Description

The `2Factor Finance` protocol has a utility `PerpVaultHelpers` contract with shared math for underlying-denominated perp vaults: deviation ratio computation, perp valuation, position sizing, and flash mint/redeem wrappers. In the process of examining its logic to flash-mint the vault's perp and return the underlying value of the minted amount, we notice the underlying value of the minted amount may use stale perp state at the pre-trade reserve rate.

To elaborate, we show below the implementation of this specific function, i.e., `flashMintPerpByValue()`. While it properly computes the minted mount, i.e., `perpMinted`, the respective underlying value is calculated from the given `perpState`, which may become stale from the the flash mint operation. In other words, the flash mint operation may trigger the so-called lazy rollover when `perpSr` has "too much" underlying liquidity. If the lazy rollover is triggered, post rollover reserve transformation is non-trivial and there is a need to refresh the post-rollover reserve data. With that, the direct use of pre-rollover reserve data may lead to inaccurate `underlyingValueMinted` calculation (line 240).

```
228   function flashMintPerpByValue(
229     IPerpController controller,
230     IPerp.ReserveState memory perpState,
231     uint256 underlyingAmtIn,
232     bool isPerpSrVault
233   ) internal returns (uint256 underlyingValueMinted, uint256 feePerc) {
234     uint256 perpMinted;
```

```
235     (perpMinted, feePerc) = flashMintPerp(
236       controller,
237       underlyingAmtIn,
238       isPerpSrVault
239     );
240     underlyingValueMinted = perpToUnderlying(perpState, perpMinted);
241   }
```

Listing 3.1: `PerpVaultHelpers::flashMintPerpByValue()`

Fortunately, the returned `underlyingValueMinted` amount is not currently used. With that, the caller of `flashMintPerpByValue()` may be revised to invoke `flashMintPerp()` instead. And it affects two functions, i.e., `UnderlyingPerpVault::rebalance()` and `UsdPerpSrVault::_rebalanceBuyUnderlying()`.

Recommendation Improve the above-mentioned routines to avoid the use of stale perp state with simplified rebalance logic.

Status This issue has been fixed by this commit: `a22ab82`.

3.2 Missing Swap Leftover Refund in UniswapV4CbBtcUsdcStrategy

- ID: PVE-002
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `PerpHelpers`
- Category: Business Logic [7]
- CWE subcategory: CWE-770 [4]

Description

The `2Factor Finance` protocol has depositor-facing vaults that make use of `DEX` strategies to facilitate the token swaps. While reviewing a single-hop `Uniswap V4`-based `DEX` strategy, we notice the possibility of holding leftover input tokens, which need to be refunded back to the user.

To elaborate, we show below the implementation of the related routine, i.e., `unlockCallback()`. The swap follows the well-defined `unlock-callback` pattern via `Uniswap V4 PoolManager`. After verifying the caller from `Uniswap V4 PoolManager`, it uncovers and rebuilds the caller and swap parameters, interacts with `V4 PoolManager` for the intended token swaps, performs token settlement, and returns the output token amount in `abi.encode(amountOut)`. It comes to our attention that the user transfers in the `amountIn` input tokens (line 211) and the swap may actually use `inputAmt` (line 227). If the input amount is not fully used, we need to refund the leftover back to the user.

```
206   function unlockCallback(
207     bytes calldata data
```

```
208   ) external override returns (bytes memory) {
209     if (msg.sender != address(poolManager)) revert UnauthorizedCall();
210
211     (bool zeroForOne, uint256 amountIn, address recipient) = abi.decode(
212       data,
213       (bool, uint256, address)
214     );
215
216     SwapParams memory params = SwapParams({
217       zeroForOne: zeroForOne,
218       amountSpecified: -int256(amountIn),
219       sqrtPriceLimitX96: zeroForOne
220         ? TickMath.MIN_SQRT_PRICE + 1
221         : TickMath.MAX_SQRT_PRICE - 1
222     });
223
224     BalanceDelta delta = poolManager.swap(_poolKey(), params, '');
225
226     Currency inputCurrency = zeroForOne ? poolKeyCurrency0 : poolKeyCurrency1;
227     uint256 inputAmt = uint256(
228       uint128(zeroForOne ? -delta.amount0() : -delta.amount1())
229     );
230     poolManager.sync(inputCurrency);
231     IERC20(Currency.unwrap(inputCurrency)).safeTransfer(
232       address(poolManager),
233       inputAmt
234     );
235     poolManager.settle();
236
237     Currency outputCurrency = zeroForOne ? poolKeyCurrency1 : poolKeyCurrency0;
238     uint256 amountOut = uint256(
239       uint128(zeroForOne ? delta.amount1() : delta.amount0())
240     );
241     poolManager.take(outputCurrency, recipient, amountOut);
242
243     return abi.encode(amountOut);
244   }
```

Listing 3.2: `UniswapV4CbBtcUsdcStrategy::unlockCallback()`

Recommendation Revise the above `unlockCallback()` routine to refund any remaining tokens, if any, back to the user.

Status This issue has been fixed by this commit: `a22ab82`.

3.3 Improved Validation of Function Arguments

- ID: PVE-003
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `UsdPerpSrVault`
- Category: Coding Practices [6]
- CWE subcategory: CWE-1126 [1]

Description

The `2Factor Finance` protocol has two user-facing vaults – `UsdPerpSrVault` and `UnderlyingPerpVault`. Both vaults have a bootstrap safeguard mechanism in place so that the very fist deposit or the seed deposit must come from the trusted owner account.

In the following, we show the implementation of the seed deposit, i.e., `initDeposit()`, from the `UsdPerpSrVault` contract. The seed deposit is necessary to mitigate possible inflation attacks against the vault. However, it comes to our attention that the given initial token amount may be strengthened to be no smaller than the respective unit amount `usdUnitAmt`, instead of current non-zero requirement. Further, instead of minting the computed `notesMinted` to the seed depositor, we may adjust the share amount to `notesMinted - MINIMUM_LIQUIDITY` (line 434). Note the same suggestion also applies to the `UnderlyingPerpVault` contract.

```
418   function initDeposit(
419     uint256 usdAmt
420   )
421     external
422     onlyOwner
423     whenUnseeded
424     nonReentrant
425     whenNotPaused
426     returns (uint256 notesMinted)
427   {
428     if (usdAmt <= 0) return 0;

430     notesMinted = _usdToVal(usdAmt);

432     usd.safeTransferFrom(msg.sender, address(this), usdAmt);
433     _mint(address(this), MINIMUM_LIQUIDITY);
434     _mint(msg.sender, notesMinted);

436     emit Deposit(usdAmt, 0, notesMinted);
437   }
```

Listing 3.3: `UsdPerpSrVault::initDeposit()`

Recommendation Strengthen the above-mentioned routines to ensure proper seed deposit amount is given to initialize the vault.

Status This issue has been fixed by this commit: a22ab82.

3.4 Improved Precision By Multiplication And Division Reordering

- ID: PVE-004
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: UsdPerpSrVault
- Category: Numeric Errors [8]
- CWE subcategory: CWE-190 [2]

Description

SafeMath is a widely-used Solidity math library that is designed to support safe math operations by preventing common overflow or underflow issues when working with uint256 operands. While it indeed blocks common overflow or underflow issues, the lack of float support in Solidity may introduce another subtle, but troublesome issue: precision loss. In this section, we examine one possible precision loss source that stems from the different orders when both multiplication (mul) and division (div) are involved.

In particular, we use the whenUnderlyingHealthy() modifier from the UsdPerpSrVault contract as an example. This modifier is used to revert the transaction when when the reference price sourced from dexStrategy diverges from the underlying oracle by more than maxUnderlyingDeviationPerc.

```
222  modifier whenUnderlyingHealthy() {
223    uint256 oraclePrice = perpController.underlyingOracle().getValidPrice();
224    uint256 refPrice = dexStrategy.referencePrice(underlying, usd);
225    uint256 deviation = refPrice > oraclePrice
226      ? refPrice - oraclePrice
227      : oraclePrice - refPrice;
228    if (deviation.mulDiv(ONE, oraclePrice) > maxUnderlyingDeviationPerc) {
229      revert RebalanceUnavailable();
230    }
231    _;
232  }
```

Listing 3.4: UsdPerpSrVault::whenUnderlyingHealthy()

We notice the deviation calculation (line 228) involves mixed multiplication and devision. For improved precision, it is better to calculate the deviation comparison as follows: if (deviation * ONE > maxUnderlyingDeviationPerc * oraclePrice). Note that the resulting precision loss may be just a small number, but it plays a critical role when certain boundary conditions are met. And it is always the preferred choice if we can avoid the precision loss as much as possible. Note the UsdPerpSrVault::_checkRateFloor() and BaseDexStrategy::maxFillable() routines can be similarly improved.

Recommendation Revise the above calculations to better mitigate possible precision loss.

Status This issue has been fixed by this commit: a22ab82.

3.5 Revisited Tranche Set Management Logic in PerpTranche

- ID: PVE-005
- Severity: Informational
- Likelihood: N/A
- Impact: N/A
- Target: PerpJr
- Category: Business Logic [7]
- CWE subcategory: CWE-770 [4]

Description

As mentioned earlier, 2Factor Finance creates PerpSr and PerpJr tranches from underlying assets, wrapped as perpetual ERC20 notes with automatic rollover and rebalancing mechanics. Each tranche reserves hold a mix of tranches and underlying tokens. During rollover, PerpJr receives stale assets from PerpSr and uses its underlying to mint fresh tranches. The newly minted senior tranches are sent back to PerpSr while juniors remain here. While analyzing the rollover mechanism, we notice the existing approach to manage to current set of tranches may be improved.

In the following, we show the implementation of the related rollover function, i.e., bondDepositsForRollover(). In essence, it deposits underlying into bonds to mint fresh tranches during rollover and synchronizes the set of tranches with syncTranche() (lines 65-66). It comes to our attention that the call to syncTranche() may be replaced with _syncTranche() as the attached modifier of onlyController is already duplicated in bondDepositsForRollover(). This replacement can further make the syncTranche() function redundant as syncTranche() requires the caller to be the PerpController contract that does not call this syncTranche() function at all.

```
58  function bondDepositsForRollover(
59    IBondsManager manager,
60    IBondsManager.BondDepositParams[] memory bondDeposits
61  ) external onlyController {
62    for (uint256 i = 0; i < bondDeposits.length; ++i) {
63      (TokenAmount memory srAmtMint, TokenAmount memory jrAmtMint) = manager
64        .deposit(bondDeposits[i].bondID, bondDeposits[i].underlyingAmt);
65      syncTranche(srAmtMint.token.asTranche());
66      syncTranche(jrAmtMint.token.asTranche());
67    }
68    logBalance(underlying());
69  }
```

Listing 3.5: PerpJr::bondDepositsForRollover()

Recommendation Improve the above-mentioned routine to remove the redundant `syncTranche()` function .

Status This issue has been fixed by this commit: `a22ab82`.

3.6 Trust Issue of Admin Keys

- ID: PVE-006
- Severity: Medium
- Likelihood: Low
- Impact: Medium
- Target: `Multiple Contracts`
- Category: Security Features [5]
- CWE subcategory: CWE-287 [3]

Description

In the `Fragments` protocol, there is a special administrative account, i.e., `owner`. This `owner` account plays a critical role in governing and regulating the system-wide operations (e.g., configure various settings, assign keepers, and execute privileged operations). It also has the privilege to control or govern the flow of assets within the protocol contracts. In the following, we examine the privileged account and the related privileged accesses in current contracts.

```
227   function updateKeeper(address keeper_) public onlyOwner {...}

229   function updateTargetDR(uint256 targetDR_) public onlyOwner {...}

231   function updateSwapSlippagePerc(uint256 perc) public onlyOwner {...}

233   function updateDexStrategy(
234     IDexStrategy newStrategy) public onlyOwner whenStrategyHealthy(newStrategy) {...}

236   function updateRebalanceTimingParams(
237     uint256 freqSec_,
238     uint256 offsetSec_) public onlyOwner {...}

240   function updateMinSwapVal(uint256 minVal) public onlyOwner {...}

242   function updateMaxFlashFeePerc(uint256 perc) public onlyOwner {...}

244   function updateMaxDeploymentVelocityPerc(uint256 perc) public onlyOwner {...}

246   function updateUsdOracle(
247     IPriceOracle usdOracle_,
248     uint256 maxDeviation_) public onlyOwner {...}

250   function updateMaxUnderlyingDeviationPerc(uint256 perc) public onlyOwner {...}
```

Listing 3.6: Example Privileged Operations in `UnderlyingPerpVault`

We understand the need of the privileged functions for proper contract operations, but at the same time the extra power to these privileged accounts may also be a counter-party risk to the contract users. Therefore, we list this concern as an issue here from the audit perspective and highly recommend making these privileges explicit or raising necessary awareness among protocol users.

In the meantime, the audited protocol makes use of the proxy contracts to allow for future upgrades. The upgrade is a privileged operation and the management of the related admin key also falls in this trust issue.

Recommendation Promptly transfer the privileged account to the intended DAO-like governance contract. All changes to privileged operations may need to be mediated with necessary timelocks. Eventually, activate the normal on-chain community-based governance life-cycle and ensure the intended trustless nature and high-quality distributed governance.

Status The issue has been resolved as the team clarifies the use of a publicly documented multisig address as the `owner` account.

PeckShield

4 Conclusion

In this audit, we have analyzed the design and implementation of the 2Factor Finance protocol for perpetual tranches. The protocol creates senior and junior tranches from underlying assets, wrapped as perpetual ERC20 notes with automatic rollover and rebalancing mechanics. It enables users to gain leveraged or de-leveraged exposure to underlying assets through a two-tranche bond system. Users can mint perpetual senior notes for lower-volatility exposure or perpetual junior notes for leveraged exposure, backed by dynamically managed reserves of tranched bonds. The current code base is clearly organized and those identified issues are promptly confirmed and fixed.

Meanwhile, we need to emphasize that Solidity-based smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

PeckShield

References

[1] MITRE. CWE-1126: Declaration of Variable with Unnecessarily Wide Scope. https://cwe.mitre.org/data/definitions/1126.html.

[2] MITRE. CWE-190: Integer Overflow or Wraparound. https://cwe.mitre.org/data/definitions/190.html.

[3] MITRE. CWE-287: Improper Authentication. https://cwe.mitre.org/data/definitions/287.html.

[4] MITRE. CWE-770: Allocation of Resources Without Limits or Throttling. https://cwe.mitre.org/data/definitions/770.html.

[5] MITRE. CWE CATEGORY: 7PK - Security Features. https://cwe.mitre.org/data/definitions/254.html.

[6] MITRE. CWE CATEGORY: Bad Coding Practices. https://cwe.mitre.org/data/definitions/1006.html.

[7] MITRE. CWE CATEGORY: Business Logic Errors. https://cwe.mitre.org/data/definitions/840.html.

[8] MITRE. CWE CATEGORY: Numeric Errors. https://cwe.mitre.org/data/definitions/189.html.

[9] MITRE. CWE VIEW: Development Concepts. https://cwe.mitre.org/data/definitions/699.html.

[10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

[11] PeckShield. PeckShield Inc. https://www.peckshield.com.

PeckShield